

PERANCANGAN SISTEM AUTHSHIELD MENGGUNAKAN ALGORITMA BCrypt DAN ENTROPI SHANNON UNTUK KEAMANAN AUTENTIKASI WEB

Danang Adiwijaya ¹⁾, Kanda Rifqi Alfaz ²⁾, M Khalid Al Rejeki ³⁾

¹⁾ Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: danangadiwijaya977@gmail.com, kandarifqi3@gmail.com, halidrizki54@gmail.com

Abstrak

Keamanan data autentikasi seperti kata sandi merupakan prinsip utama dalam pengembangan aplikasi berbasis web di era transformasi digital. Kebocoran basis data akibat celah siber sering kali diperparah oleh penggunaan fungsi hashing tradisional yang cepat serta rendahnya tingkat entropi kata sandi pengguna. Penelitian ini bertujuan untuk merancang sebuah sistem autentikasi berlapis bernama AuthShield yang mampu memberikan perlindungan kokoh pada kredensial pengguna. Metode yang diusulkan mengintegrasikan dua lapisan pertahanan utama. Lapisan pertama adalah validasi preventif di sisi client/backend menggunakan pendekatan teori entropi Shannon untuk memastikan kekuatan acak dari kata sandi sebelum disimpan. Lapisan kedua adalah implementasi algoritma kriptografi Bcrypt yang dimodifikasi melalui penggabungan tiga komponen, yaitu kata sandi, Salt unik, dan Secret Key. Berbeda dengan Salt yang disimpan di dalam basis data, Secret Key diisolasi secara terpisah pada level environment variable server aplikasi. Hasil perancangan arsitektur ini menunjukkan bahwa alur registrasi dan login AuthShield mampu mengeliminasi single point of failure. Dengan memisahkan komponen Secret Key dari basis data, sistem ini secara teoretis memastikan bahwa data kata sandi yang bocor tetap tidak dapat direkonstruksi atau dipatahkan melalui serangan rainbow table maupun offline brute-force.

Kata kunci: Bcrypt; Entropi Shannon; Keamanan Web; Secret Key.

Abstract

Authentication data security, such as passwords, represents a fundamental principle in web-based application development amid the current digital transformation era. Database breaches resulting from cyber vulnerabilities are often exacerbated by the use of fast traditional hashing functions and low user password entropy. This study aims to design a multi-layered authentication system named AuthShield capable of providing robust protection for user credentials. The proposed method integrates two primary defense layers. The first layer is a preventive validation on the client/backend side using Shannon's entropy theory approach to ensure the structural strength of the password before storage. The second layer is the implementation of the Bcrypt cryptographic algorithm, modified by combining three components: the password, a unique Salt, and a Secret Key. Unlike the Salt which is stored within the database, the Secret Key is isolated separately at the application server's environment variable level. The architectural design outcomes demonstrate that AuthShield's registration and login workflows effectively eliminate the single point of failure. By segregating the Secret Key component from the database, this system theoretically ensures that leaked password data remains resilient against reconstruction or cracking via rainbow table and offline brute-force attacks.

Keyword: Bcrypt; Secret Key; Shannon Entropy; Web Security.

I. PENDAHULUAN

Dalam era transformasi digital saat ini, keamanan data pengguna telah menjadi prinsip utama dalam pengembangan aplikasi berbasis web. Di antara berbagai jenis data sensitif, data autentikasi seperti password merupakan target utama bagi para peretas karena berfungsi untuk mengakses hak istimewa pengguna. Kegagalan dalam melindungi integritas data password seringkali berakar pada penggunaan algoritma

hashing tradisional yang cepat atau penyimpanan data dalam bentuk plaintext [1].

Kerentanan sistem autentikasi ini sangat dipengaruhi oleh tingkat kekuatan kata sandi yang dibuat oleh pengguna, yang dalam ilmu kriptografi diukur melalui tingkat entropi [2]. Entropi adalah ukuran seberapa acak, tidak tertebak, dan bervariasi sebuah kata sandi berdasarkan panjang dan ragam karakter yang digunakan. Kata sandi dengan entropi yang tinggi

akan memperluas ruang pencarian kombinasi sehingga membutuhkan waktu yang lama bagi peretas untuk menebaknya.

Masalah entropi rendah ini diperparah oleh penggunaan algoritma hashing lama seperti MD5 atau SHA-1 yang awalnya dirancang hanya untuk memeriksa integritas data dengan performa super cepat. Di era komputasi modern yang didukung oleh Graphics Processing Unit berperforma tinggi, kecepatan ini justru menjadi celah keamanan yang fatal karena penyerang dapat menguji miliaran kombinasi hash per detik. Akibatnya, kata sandi pengguna yang berentropi rendah dapat dipatahkan hanya dalam hitungan detik melalui serangan brute-force atau pencocokan dengan basis data rainbow table [3].

Dalam menghadapi tantangan tersebut, National Institute of Standards and Technology (NIST) menegaskan bahwa mekanisme autentikasi modern harus menggunakan fungsi yang sengaja dirancang lambat (computationally expensive) [4]. Salah satu skema kriptografi yang memenuhi standar ini adalah algoritma Bcrypt [5]. Keunggulan utama Bcrypt terletak pada parameter cost factor yang memungkinkan pengembang untuk memperlambat waktu pemrosesan hashing secara sengaja. Efek dari cost factor secara langsung mengatasi kelemahan kata sandi berentropi rendah dengan memaksa komputer penyerang bekerja berkali-kali lipat lebih keras dan lama untuk menguji satu kombinasi kata sandi. Meskipun demikian, ketergantungan penuh pada data yang tersimpan di dalam basis data masih menyisakan single point of failure jika peretas berhasil menembus basis data melalui celah SQL Injection dan melakukan serangan secara offline.

Untuk mengurangi risiko kebocoran basis data tersebut, diperlukan sebuah lapisan pertahanan berlapis melalui proyek perancangan sistem registrasi dan login yang dinamakan AuthShield. AuthShield dikembangkan untuk melampaui implementasi standar Bcrypt dengan menerapkan gabungan penggunaan Salt unik dengan Secret Key yang dibatasi pada level environment variable web backend. Penambahan Secret Key bertindak sebagai penyedia entropi eksternal yang kuat pada kata sandi sebelum diproses oleh Bcrypt. Melalui arsitektur AuthShield ini, jika terjadi situasi buruk di mana basis data bocor, peretas tetap tidak akan bisa

melakukan rekonstruksi atau brute-force karena tidak memiliki komponen Secret Key yang disimpan terpisah di luar basis data tersebut.

II. TINJAUAN PUSTAKA

Landasan utama dalam membangun sistem keamanan autentikasi berbasis web berakar pada penerapan fungsi hash kriptografi searah. Secara fundamental, fungsi hash kriptografi adalah sebuah algoritma matematis yang menerima masukan data dengan panjang variabel dan mengubahnya menjadi representasi string berukuran tetap yang unik. Dalam konteks perlindungan kredensial, proses ini harus bersifat satu arah yang berarti nilai hash yang dihasilkan tidak dapat didekripsi atau dikembalikan lagi ke bentuk plaintext aslinya [2]. Oleh karena itu, validasi identitas pengguna tidak dilakukan dengan cara membongkar kembali kata sandi yang tersimpan melainkan dengan merekonstruksi ulang proses hashing pada kata sandi yang dimasukkan saat login lalu mencocokkan hasilnya dengan nilai hash yang telah berada di dalam basis data.

Kekuatan awal dari sebuah kata sandi sebelum memasuki proses hashing ditentukan oleh ukuran matematis dari tingkat keacakan dan prediktabilitasnya yang dikenal sebagai entropi kata sandi. Evaluasi nilai entropi merupakan langkah preventif yang krusial untuk mengukur seberapa tahan sebuah kata sandi terhadap serangan tebakan acak. Kalkulasi tingkat keacakan ini dihitung menggunakan rumus Shannon untuk entropi informasi, yaitu $E = L \times \log_2(R)$.

Dalam formula tersebut, variabel E merepresentasikan nilai entropi dalam satuan bit, sedangkan L menunjukkan panjang atau jumlah karakter dari kata sandi, dan R merupakan ukuran pool karakter atau ragam jenis karakter yang digunakan, seperti kombinasi huruf kecil, huruf besar, angka, dan simbol [2]. Semakin tinggi nilai entropi (E), maka ruang pencarian kombinasi yang harus dipecahkan oleh penyerang akan meluas secara eksponensial sebesar 2^E . Sebaliknya, jika pengguna menciptakan kata sandi dengan pola yang mudah ditebak, nilai entropinya akan sangat rendah sehingga sistem memerlukan mekanisme pertahanan tambahan di

sisi backend untuk memperlambat laju komputasi peretas.

Untuk menambal kelemahan kata sandi yang berentropi rendah tersebut, standar keamanan modern beralih menggunakan fungsi turunan kunci yang bersifat adaptif, salah satunya adalah algoritma Bcrypt yang dikembangkan berdasarkan cipher Blowfish [5]. Keunggulan utama Bcrypt dibandingkan algoritma lama seperti MD5 atau SHA-256 terletak pada sifatnya yang secara inheren sengaja dirancang berjalan lambat. Parameter kunci yang mengatur perlambatan komputasi ini disebut sebagai cost factor atau work factor. Hubungan antara cost factor dengan jumlah iterasi internal yang dilakukan oleh Bcrypt dirumuskan secara eksponensial yaitu Iterasi = 2^{cost} .

Melalui rumus tersebut, setiap kenaikan satu digit nilai cost factor akan melipatgandakan jumlah kerja komputasi dan waktu yang diperlukan untuk menghasilkan satu nilai hash. Karakteristik adaptif ini memungkinkan sistem untuk mengimbangi peningkatan performa perangkat keras komputer di masa depan; pengembang cukup menaikkan nilai parameter cost secara berkala untuk memastikan bahwa upaya serangan brute-force secara massal tetap membutuhkan biaya komputasi dan waktu yang tidak masuk akal bagi peretas [5].

Kendati Bcrypt sangat tangguh dalam memperlambat serangan, algoritma ini tetap membutuhkan komponen pengamanan tambahan berupa Salt dan Secret Key untuk mencapai tingkat keamanan yang optimal. Salt adalah string acak unik yang digenerasikan secara otomatis untuk setiap pengguna dan berfungsi untuk memastikan bahwa dua kata sandi yang sama akan menghasilkan nilai hash yang sepenuhnya berbeda, sehingga secara efektif menggagalkan serangan berbasis rainbow table atau kamus pra-komputasi [3].

Namun, karena Salt wajib disimpan di dalam basis data bersama dengan nilai hash agar dapat digunakan kembali pada proses verifikasi, ketergantungan ini memunculkan

risiko tunggal jika basis data tersebut bocor. Guna memitigasi celah tersebut, penggunaan Secret Key menjadi proteksi lapis kedua yang sangat direkomendasikan [4]. Berbeda dengan Salt, Secret Key tidak ikut disimpan di dalam basis data melainkan diisolasi di luar sistem, biasanya pada level konfigurasi server atau environment variable aplikasi. Dengan menggabungkan seluruh komponen tersebut ke dalam formula kriptografi sebelum diproses oleh Bcrypt, kredensial pengguna akan tetap terlindungi secara kokoh meskipun peretas berhasil mengunduh seluruh isi basis data, karena peretas tidak memiliki Secret Key eksternal yang diperlukan untuk merekonstruksi hash secara luring.

III. METODOLOGI PENELITIAN

Penelitian ini dilakukan melalui beberapa tahapan yang saling berkaitan dalam pengembangan sistem autentikasi berlapis AuthShield. Kegiatan diawali dengan studi literatur untuk mengidentifikasi kelemahan fungsi hash yang umum digunakan serta menelaah standar komputasi adaptif yang direkomendasikan oleh NIST. Hasil kajian tersebut menjadi dasar dalam merancang arsitektur sistem yang memisahkan penyimpanan komponen keamanan, sehingga risiko kegagalan akibat kebocoran pada satu basis data dapat diminimalkan. Setelah tahap perancangan selesai, rancangan yang telah disusun diimplementasikan ke dalam bentuk purwarupa menggunakan bahasa pemrograman PHP dengan framework Laravel 11 sebagai platform pengembangan backend.

Penerapan arsitektur AuthShield diwujudkan secara langsung pada proses registrasi pengguna. Ketika pengguna memasukkan kata sandi melalui antarmuka aplikasi, data tersebut terlebih dahulu diperiksa oleh backend sebelum disimpan ke dalam sistem. Pada tahap ini, dilakukan validasi menggunakan pendekatan entropi Shannon untuk mengukur tingkat kekuatan kata sandi berdasarkan panjang, tingkat keacakan, dan keragaman karakter yang digunakan. Apabila hasil perhitungan menunjukkan nilai entropi berada di bawah batas minimum yang telah ditentukan, yaitu 50 bit,

proses registrasi tidak dapat dilanjutkan. Pengguna kemudian diminta untuk membuat kombinasi kata sandi yang lebih kuat agar memenuhi persyaratan keamanan yang ditetapkan.

Jika kata sandi yang dimasukkan telah memenuhi batas minimum entropi sebesar 50 bit, proses dilanjutkan ke tahap berikutnya, yaitu pembentukan nilai hash menggunakan algoritma Bcrypt yang telah dimodifikasi. Pada tahap ini, sistem menghasilkan nilai *salt* yang bersifat acak dan unik untuk setiap pengguna. Selain itu, sistem juga mengambil *secret key* yang disimpan pada berkas konfigurasi lingkungan aplikasi (*environment variable* atau *.env*), sehingga terpisah dari basis data utama. Selanjutnya, kata sandi pengguna, nilai *salt*, dan *secret key* tersebut digabungkan terlebih dahulu sebelum diproses menggunakan fungsi Bcrypt untuk menghasilkan nilai hash yang akan disimpan.

Pembentukan nilai hash dilakukan menggunakan algoritma Bcrypt dengan *cost factor* sebesar 12. Parameter ini dipilih untuk meningkatkan kompleksitas proses komputasi melalui penambahan jumlah iterasi, sehingga waktu yang diperlukan untuk menghasilkan hash menjadi lebih lama dan lebih sulit dieksploitasi melalui serangan *brute force*. Setelah proses hashing selesai, nilai hash bersama *salt* pengguna disimpan ke dalam basis data, sedangkan *secret key* tetap disimpan pada berkas konfigurasi server dan tidak disertakan dalam penyimpanan data pengguna. Pada proses autentikasi, mekanisme yang digunakan mengikuti tahapan yang serupa. Ketika pengguna memasukkan kata sandi, sistem mengambil nilai *salt* yang terkait dengan akun tersebut dari basis data, kemudian menggabungkannya dengan *secret key* yang tersimpan pada *environment variable*. Kombinasi tersebut diproses kembali menggunakan Bcrypt dengan parameter yang sama untuk menghasilkan nilai hash pembandingan. Selanjutnya, hasil yang diperoleh dicocokkan dengan nilai hash yang tersimpan di basis data guna menentukan validitas kredensial yang diberikan oleh pengguna.

IV. HASIL DAN PEMBAHASAN

Tahap implementasi menghasilkan sebuah tampilan sistem autentikasi berlapis AuthShield yang dikembangkan menggunakan

bahasa pemrograman PHP 8.x dan framework Laravel 11. Perlindungan berlapis dihasilkan secara konkret melalui isolasi *Secret Key* pada berkas konfigurasi lingkungan (*.env*) aplikasi. Berkas konfigurasi secara default dipertahankan di dalam lingkungan server dan diisolasi agar tidak masuk ke dalam penyimpanan basis data utama maupun repositori kode sumber. Pada modul registrasi dan login, alur kerja *backend* mengintegrasikan pustaka perhitungan entropi Shannon untuk mengevaluasi kekuatan kata sandi pengguna secara langsung. Melalui pendekatan ini, sistem menerapkan validasi preventif yang akan menolak pembuatan kata sandi jika hasil perhitungan strukturalnya berada di bawah nilai ambang batas keamanan yang ditentukan, yaitu 50 bit. Setelah kata sandi dinyatakan lolos dari pengujian entropi, proses pengamanan tingkat lanjut dilakukan dengan memproses kredensial tersebut menggunakan fungsi turunan kunci adaptif Bcrypt yang dikombinasikan bersama komponen *Salt* unik dan *Secret Key* sebelum disimpan secara persisten.

```
C:\VackLangit\Criptonode>bekrip.js
Cost Factor: 7
Salt      : $2b$07$/8S9WkXx8ENq298bx2270
Hash      : $2b$07$/8S9WkXx8ENq298bx2270XxE6p1q109eGQ.2pPQs9sIFDXR8.f0m
Waktu     : 7.56 ms

-----
Cost Factor: 8
Salt      : $2b$08$uIGrm7CT0bGKQX.NTzRQ20
Hash      : $2b$08$uIGrm7CT0bGKQX.NTzRQ20jwEUNIOYqr0TOMuyCR3K93gn5E4ETu
Waktu     : 13.36 ms

-----
Cost Factor: 9
Salt      : $2b$09$zkjCtqHgy.xcPhyE5a47Be
Hash      : $2b$09$zkjCtqHgy.xcPhyE5a47BeRe2chew57XSwu7i4G.0BwgTqeegglce
Waktu     : 27.06 ms

-----
Cost Factor: 10
Salt      : $2b$10$8BH3kwoRXdKcf8d9il.jve
Hash      : $2b$10$8BH3kwoRXdKcf8d9il.jve/D4ltC0NkRhC5TRbrubXQ26j8AYX/qe
Waktu     : 52.75 ms

-----
Cost Factor: 11
Salt      : $2b$11$sgafpl.bccSBendIyYR0Cu
Hash      : $2b$11$sgafpl.bccSBendIyYR0CuZwJ00D8MheSCNcwj07lUv3A0knZAK
Waktu     : 101.34 ms

-----
Cost Factor: 12
Salt      : $2b$12$ZP0ccR1WvAhimVogDjNe
Hash      : $2b$12$ZP0ccR1WvAhimVogDjNe1rXwUW60kmPL5x1LP44bgr0Isa28y
Waktu     : 260.44 ms

-----
Cost Factor: 13
Salt      : $2b$13$Sot.b/qy58/PhoQZHP5fo0
Hash      : $2b$13$Sot.b/qy58/PhoQZHP5fo0N034eHc7sIwCNCQf2oWiaz4rj1Kw8YK
Waktu     : 398.59 ms

-----
Cost Factor: 14
Salt      : $2b$14$a92PHBvQkgCU0jy7oZ5bru
Hash      : $2b$14$a92PHBvQkgCU0jy7oZ5bruSr635EysE2mV2/7bp1W8ZmyiK1ce
Waktu     : 829.77 ms
```

Gambar 1. Pengujian Cost Factor

Untuk menentukan konfigurasi parameter terbaik pada lingkungan server backend AuthShield, pengujian performa komputasi dilakukan secara empiris dengan mengukur durasi waktu eksekusi *hashing* searah pada rentang *cost factor* dari tingkatan 7 hingga 14.

Rekaman data pengujian ini ditunjukkan pada **Gambar 1**. Berdasarkan log terminal pada gambar tersebut, hasil eksekusi membuktikan secara nyata sifat pertumbuhan eksponensial dari arsitektur komputasi Bcrypt yang selaras dengan rumus matematika hubungan antara *cost factor* dengan jumlah iterasi internal yang dirumuskan secara eksponensial yaitu Iterasi = 2^{cost} . Setiap kenaikan satu tingkat kesulitan *cost factor* terbukti secara konsisten melipatgandakan jumlah kerja komputasi serta durasi waktu respons yang diperlukan server untuk memproses sebuah kata sandi.

Pada tingkat kesulitan terendah yaitu *cost factor* 7, pemrosesan berjalan sangat cepat dan hanya membutuhkan waktu sekitar 7,56 ms, namun kecepatan respons ini dinilai sangat rentan terhadap ancaman perangkat keras modern. Durasi tersebut melonjak secara bertahap seiring bertambahnya angka tingkatan kesulitan, mulai dari *cost factor* 11 yang membutuhkan waktu 101,34 ms, *cost factor* 12 dengan waktu 200,44 ms, hingga mencapai puncaknya pada *cost factor* 14 yang menghabiskan waktu komputasi yang sangat lama sebesar 829,77 ms. Berdasarkan standar kenyamanan pengguna aplikasi berbasis web, batas toleransi waktu tunggu respons halaman login yang ideal agar tidak menyebabkan *lagging* adalah di bawah 500 ms. Oleh karena itu, hasil pengujian ini memberikan landasan krusial bagi tim untuk menyimpulkan bahwa penggunaan konfigurasi *cost factor* 12 merupakan parameter bawaan yang paling optimal untuk AuthShield. Pilihan rentang ini dinilai berhasil mencapai *trade-off* terbaik karena mampu menyajikan ketahanan komputasi ekstra yang mempersulit upaya penyerangan massal tanpa mengorbankan performa respons kecepatan server *backend* dalam melayani pengguna sah.

Request #	Endpoint	Status code	Response content	Time	Server	Length	Comment
0	StartUpPage	200	2048	1481			
1	AuthShield	200	2048	1481			
2	AuthShield	200	2048	1481			
3	AuthShield	200	2048	1481			
4	AuthShield	200	2048	1481			
5	AuthShield	200	2048	1481			
6	AuthShield	200	2048	1481			
7	AuthShield	200	2048	1481			
8	AuthShield	200	2048	1481			
9	AuthShield	200	2048	1481			
10	AuthShield	200	2048	1481			
11	AuthShield	200	2048	1481			
12	AuthShield	200	2048	1481			
13	AuthShield	200	2048	1481			
14	AuthShield	200	2048	1481			
15	AuthShield	200	2048	1481			
16	AuthShield	200	2048	1481			
17	AuthShield	200	2048	1481			
18	AuthShield	200	2048	1481			
19	AuthShield	200	2048	1481			
20	AuthShield	200	2048	1481			
21	AuthShield	200	2048	1481			

Gambar 2. Uji Keamanan Brute Force

Pengujian keamanan dilakukan untuk memvalidasi efektivitas gabungan komponen *Password*, *Salt*, dan *Secret Key* pada AuthShield dari ancaman *database leakage*. Simulasi serangan dilancarkan secara *offline* menggunakan

bantuan perangkat lunak *Burp Suite* melalui metode *Dictionary Attack* dengan memanfaatkan puluhan *payload* kata sandi yang bervariasi. Target pengujian diarahkan pada pemecahan nilai hash kredensial hasil ekstraksi data yang disimulasikan bocor, di mana rekaman log dari serangan interpolasi otomatis tersebut dipaparkan secara detail pada Gambar 2. Analisis mendalam terhadap log hasil simulasi serangan pada gambar tersebut menunjukkan bahwa seluruh *request payload* kata sandi uji coba dikembalikan oleh sistem dengan kode status HTTP 302 yang menandakan adanya redireksi penolakan akses secara instan.

Meskipun penyerang mencoba memasukkan variasi kata sandi dasar milik pengguna secara tepat seperti pada *payload @Kanda23* yang memicu perbedaan ukuran respons teknis menjadi 1511 bita sistem AuthShield tetap berhasil menggagalkan upaya autentikasi tersebut. Hasil ini membuktikan keandalan mutlak dari skema pertahanan berlapis yang dirancang ketika situasi buruk terjadi di mana basis data berhasil ditembus, penyerang hanya mampu mengunduh data *username*, string *Salt*, dan nilai hash Bcrypt. Karena komponen *Secret Key* diisolasi secara ketat di luar basis data pada tingkat *environment variable* server aplikasi, penyerang yang melakukan serangan secara *offline* tidak memiliki komponen eksternal krusial tersebut untuk menyusun kembali formula perangkajian dasar yaitu *Password* + *Salt* + *Secret Key*. Dampaknya, pembuatan *rainbow table* khusus maupun tebakan teoretis otomatis melalui *brute-force* menjadi buntu dan gagal total karena ruang pencarian kombinasi dikunci rapat oleh *Secret Key* yang tidak dapat diakses dari data yang bocor.

V. KESIMPULAN DAN SARAN

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem AuthShield berhasil memberikan perlindungan berlapis yang kokoh untuk mengamankan data autentikasi pada aplikasi berbasis web. Pengintegrasian teori entropi Shannon terbukti efektif sebagai lapisan pertahanan pertama di sisi backend untuk memastikan setiap kata sandi baru yang dibuat oleh pengguna memiliki tingkat keacakan struktural yang kuat sebelum memasuki

proses enkripsi. Selain itu, penerapan algoritma Bcrypt yang dikombinasikan secara unik bersama komponen *Salt* dan *Secret Key* berhasil mengeliminasi single point of failure pada basis data. Melalui pemisahan komponen *Secret Key* yang diisolasi secara ketat pada level environment variable server, AuthShield secara teoretis dan empiris mampu melumpuhkan serangan *offline brute-force* maupun *dictionary attack* menggunakan *Burp Suite*, meskipun seluruh isi basis data telah mengalami kebocoran. Dari pengujian performa komputasi, penggunaan nilai cost factor 12 disimpulkan sebagai konfigurasi paling parameter bawaan yang paling optimal karena berhasil memenuhi batas toleransi kenyamanan respons aplikasi web di bawah 500 ms sekaligus menyajikan beban kerja eksponensial yang tinggi bagi pihak penyerang. Untuk pengembangan penelitian selanjutnya, disarankan agar sistem AuthShield ini dapat diintegrasikan dengan mekanisme rate limiting atau sistem *Multi-Factor Authentication* guna mengantisipasi serangan *online brute-force* yang agresif pada gerbang *login* utama.

DAFTAR PUSTAKA

- [1] P. C. v. Oorschot, Computer Security and the Internet: Tools and Jewels. Springer, 2012.
- [2] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Pearson, 2017.
- [3] J. Kelsey, B. Schneier, C. Hall, and D. Wagner, "Secure applications of low-entropy keys," in Information Security: First International Workshop ISW'97, Springer, 1998.
- [4] W. E. Burr, D. F. Dodson, and E. M. Newton, "Electronic Authentication Guideline," National Institute of Standards and Technology (NIST), Special Publication 800-63B, 2017.
- [5] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proceedings of the 1999 USENIX Annual Technical Conference (ATC), 1999.
- [6] D.E. Winterbone. (1997). *Advanced thermodynamics for engineers*. [Online]. Tersedia di: www.knovel.com [6 Jan. 2011].
- [7] A. Paul. (1987, Okt.). "Electrical properties of flying machines". *Flying Machines*. [On-line]. 38(1), hlm. 778-998. Tersedia di: www.flyingmachjourn/properties/fly.edu [1 Des 2003].
- [8] N. Pakvilai. (2013). "Plant macronutrient analysis of enzyme ionic plasma and organic fertilizer from biodegradable waste." *The 2nd Annual South East Asian International Seminar (ASAIS)*. [On-line].hlm. 1-6. Tersedia di: <http://asais-pnj.org> [17 Jan 2014].
- [9] M. Duncan. "Engineering Concepts on Ice." Internet: www.iceengg.edu/staff.html, 25 Okt. 2000 [29 Nov. 2003].
- [10] T. Pangaribuan. "Perkembangan Kompetensi Kewacanaan di LPTK." Disertasi Doktor. IKIP Malang, Malang, 1992.
- [11] S. Maw. Bahan Kuliah, Engg 251. Topik: "Speed skating." ICT 224, Faculty of Engineering, University of Calgary, Calgary, Alberta, 31 Okt. 2003.